

Phantom Dependencies: Ghost Busting

Antonio Cyé Angelberg Kambiré
Stanford University

Julián Rodríguez Cárdenas
Stanford University

Yasin Yousuf
Stanford University

Abstract

As programming languages and the projects built with them become more complex, programmers have started to rely on centralized open source package repositories to provide building blocks for their tasks. This has led to an intricate network of dependencies between projects, and that network has repeatedly been shown to be a source of security risk. In this work, we investigate *phantom dependencies*: external packages used in code but not explicitly acknowledged in a package’s manifest. We study phantom dependencies in the npm and PyPI ecosystems using the top 1,000 packages in each repository as seed sets. For npm, we find 118 unique externally managed phantom packages across 567 seed-closure occurrences after filtering by transitive-closure membership and maintainer family. Resolver validation shows that 18 reachable candidate rows across 5 seeds currently resolve to an affected version, and 15 strict rollback-candidate rows across 6 seeds are reachable in real installs. For PyPI, a static collector successfully analyzes 801 repositories and finds 593 packages with at least one filtered phantom candidate. Because the current PyPI analysis is not version-resolved, its vulnerability results are reported as advisory-history signals rather than current exploitability. Overall, our results show that phantom dependencies are present in both ecosystems, but that security conclusions require resolver-aware measurement and careful treatment of static-analysis false positives.

1 Introduction

Programming projects often rely on various dependencies in order to function. Generally, projects will have direct dependencies that they use directly, which in turn use other dependencies, and so on. The dependencies that the project eventually depends on are its transitive dependencies. Phantom dependencies arise when a project already has a transitive dependency on another package and later uses that package in its own code without explicitly depending on it. In JavaScript, this means that a phantom package is not listed

in `package.json` but is used through `require` or `import`. In Python, the package is imported in code but is not declared in common metadata such as `requirements.txt` or `pyproject.toml`.

This mismatch matters because manifests are the main place where developers pin, audit, and update dependencies. A package that is used but not declared is harder for the downstream maintainer to control. A basic question is therefore the extent to which increasingly dense dependency structures create hidden dependency relationships and whether those relationships correspond to security-relevant vulnerability states.

Our study targets that concrete mismatch between dependency information that is typically tracked and the set of components that a project can actually access at build and runtime. In practice, phantom dependencies can remain invisible to standard dependency declarations and therefore to the workflows developers rely on for auditing and version management. We investigate this question in npm and PyPI because of their scale and centrality to modern software development. Concretely, we ask: (1) How prevalent are externally managed phantom dependencies among heavily used packages in these repositories? (2) Which phantom dependencies have known vulnerability advisory history, and how often do current installs resolve to affected versions? (3) How many packages are susceptible to the rollback attack model in which a transitive dependency constrains the visible version of a phantom package?

This paper makes three contributions:

- A reproducible pipeline for measuring externally managed phantom dependency candidates in npm and PyPI.
- A large-scale npm study of the top 1,000 packages and their dependency closure, including explicit depcheck coverage, false-positive filters, and resolver-backed validation.
- An evaluation of rollback-style exposure that separates advisory-history signals, observed affected installs, and strict rollback candidates.

2 Background and Related Work

Prior work has shown that package ecosystems amplify supply-chain risk through dense dependency networks. Zimmermann et al. [10] quantify how installing a single npm package can extend trust to many packages and maintainers. Zerouali et al. [9] study vulnerability propagation in npm and RubyGems, while Alfadel et al. [1] provide a longitudinal analysis of vulnerabilities in Python packages. Liu et al. [5] argue that vulnerability propagation analyses need repository-specific dependency resolution semantics, a lesson that is central to our measurement.

Recent practitioner work has also popularized the phrase “phantom dependency” as a source of software bill-of-materials inaccuracy [7]. PEP 770 targets a related visibility problem by standardizing embedded SBOM documents in Python distributions [4]. Other work explores exploitability along dependency chains: ChainFuzz generates downstream proof-of-concept exploits for upstream vulnerabilities [2], and Duress studies build-time supply-chain hazards in Android resource mediation [8]. Our work fits between these threads: we measure a manifest visibility gap and ask when it translates into potential security exposure.

3 Methodology

3.1 Problem Formulation: Externally Managed Phantom Dependencies

For a package p and package name x , we call x an externally managed phantom dependency of p when all of the following hold:

1. static analysis reports that p uses x ;
2. x is not declared as a direct dependency of p ;
3. x is present in p ’s transitive dependency closure; and
4. x is not in the same inferred maintainer family as p .

The third condition separates phantom dependencies from ordinary missing dependencies: if the package is not in the closure, it is not plausibly supplied by the dependency tree. The fourth condition narrows the analysis from phantom dependencies in general to externally managed phantom dependencies. Many phantom dependencies arise within a single maintainer or organizational boundary; in such cases, release and security control may remain inside one family. We focus on cases where the phantom package is managed outside the family of the dependent project.

For npm family inference, we use package scopes such as `@babel/*`, GitHub repository owners, and the legacy family metadata in our database. For PyPI, the current collector filters obvious local modules, standard-library imports, build/test tooling, private imports, and ambiguous namespace roots such

as `google` and `azure`; it does not yet construct maintainer families with the same precision as the npm pipeline.

3.2 Seed Sets and Data Collection

Rather than attempting to analyze all packages from npm and PyPI, we construct a seed set composed of the top 1,000 most-downloaded packages in each ecosystem over a six-month window from 2025-05-11 to 2025-11-11. We focus on top packages because positive or negative security properties in these packages can have substantial downstream impact. For npm, the checked-in database contains 12,610 packages reached from the seed set. We use registry metadata to build a name-level dependency closure and Dockerized depcheck output [3] to identify used-but-missing package names. The database contains depcheck results for all 12,610 packages, but only 8,606 runs succeeded; we therefore treat all prevalence results as lower bounds with explicit coverage fields.

For PyPI, we wrote a fresh static collector that resolves source repositories from PyPI metadata, clones the repository, parses common manifest files, scans Python AST imports, and filters standard-library, local, private, build/test, and artifact-like names. Of 1,000 seeds, 801 repositories were successfully analyzed. The remaining 199 failed because no source repository was identified or the clone failed. These failures are important because several high-rank packages are monorepos or expose metadata URLs that are not direct source repository URLs.

3.3 Vulnerability Assessment

We query OSV [6] for vulnerability advisory history. For npm, we also use package versions in the local database and a resolver-backed validation pass over the security-relevant seeds. The validation installs each seed with `npm install --ignore-scripts`, checks whether the phantom package is reachable from the package that uses it under Node’s module resolution layout, records the resolved version, and checks that version against OSV affected ranges.

3.4 Assessing Susceptibility to Rollback Attack

We evaluate a rollback-style scenario in which a transitive dependency constrains the version of a phantom package that a downstream package uses. We report three increasingly strict signals:

Advisory history. The phantom package has at least one OSV advisory.

Affected-range candidate. A provider dependency range intersects an affected range for the phantom package.

Strict rollback candidate. A provider range intersects an affected advisory range and excludes every known fixed version for that advisory.

Metric	Count	Share
Seed packages	1,000	–
Packages in database	12,610	–
Successful depcheck runs	8,606	68.2%
Failed depcheck runs	4,004	31.8%
Seeds with full closure coverage	585	58.5%
Mean seed-closure depcheck coverage	–	82.1%

Table 1: npm analysis coverage. Failed depcheck runs are counted as unknown, not as clean packages.

Metric	Count
Unique externally managed phantoms	118
Total seed-closure occurrences	567
Seeds with any closure-level phantom	27
Unique root-level phantoms	107
Root-level phantom occurrences	233
Seeds with any root-level phantom	19
Unique phantoms with OSV advisory history	22
Advisory-history phantom occurrences	105

Table 2: Strict npm phantom dependency results after closure and family filtering.

For npm, we further require reachability in an actual install when reporting our most conservative rollback estimates. For PyPI, we do not claim rollback vulnerability because the current pipeline does not resolve installed versions or provider constraints.

4 Results

4.1 The npm Ecosystem

We study the top 1,000 most-downloaded npm packages together with their transitive dependency closure. For each package with source information, the pipeline uses depcheck results to identify packages used by code but missing from direct dependency declarations. Raw depcheck output contains several dominant sources of false positives, including internal packages that are not published on the registry, packages that do not appear in the dependency closure, and packages that fall under the same organizational boundary as the analyzed project. The results below report the externally managed, closure-confirmed subset.

4.1.1 RQ1: npm Prevalence

Table 1 shows the main limitation of the npm analysis. The database is complete at the metadata level, but depcheck did not succeed for every package. As a result, the counts below should be read as lower bounds over successfully analyzed packages in each seed closure.

Signal	Rows	Seeds
Affected-range candidates	177	14
Strict range candidates	26	6
Reachable affected-range candidates	53	14
Reachable strict rollback candidates	15	6
Observed affected resolved versions	18	5

Table 3: npm rollback-related signals. Rows are package/phantom/provider relationships; seeds count top-level packages.

After filtering raw depcheck output by database presence, transitive closure, and family membership, we find 118 unique externally managed phantom packages across 567 seed-closure occurrences (Table 2). Only 27/1,000 seeds have at least one closure-level externally managed phantom.

The largest phantom groups are concentrated in familiar JavaScript utility and visualization packages. The most frequent names include `d3-array` (19 occurrences), `d3-time` (17), `d3-format` (14), `d3-time-format` (14), and `lodash` (13). These are not all security problems; they are evidence that widely used packages sometimes rely on packages supplied by nearby transitive dependencies.

4.1.2 RQ2: npm Vulnerability Signals

Twenty-two unique npm phantom packages have advisory history in OSV, accounting for 105 seed-closure occurrences. The highest-severity set includes `lodash`, `@babel/traverse`, `d3-color`, `semver`, `minimatch`, `tar`, and `axios`. However, advisory history alone is not equivalent to current exploitability: a package can have past advisories while the version visible in a current install is patched.

The resolver validation makes this distinction concrete. We validated 105 security-relevant npm occurrence rows across 19 seeds. All 19 seeds installed successfully. Of the 105 rows, 102 were actually reachable from the package that used the phantom. Eighteen reachable rows across 5 seeds resolved to a version affected by at least one OSV advisory. Thus the most conservative “currently affected install” estimate is 5/1,000 seeds in this measurement snapshot.

4.1.3 RQ3: npm Rollback Exposure

Table 3 reports the rollback analysis. A loose upper bound finds 14 seeds with a provider range that admits an affected version of a phantom package. The stricter advisory-level test finds 26 rows across 6 seeds where the provider range intersects an affected range and excludes the known fixed versions for that advisory. Resolver validation shows that 15 of these strict rows are reachable in current installs, still across 6 seeds. The dominant strict cases involve `minimatch`, `lodash`, `tar`, `ws`, and `axios`.

Metric	Count
Seed packages	1,000
Successful source analyses	801
Failed/no-repo analyses	199
Packages with filtered phantom candidates	593
Unique filtered phantom candidates	1,657
Total filtered phantom occurrences	4,688
Mean candidates per successful package	5.85
Median candidates per successful package	2

Table 4: PyPI static-analysis results after artifact and tooling filters.

We deliberately describe these as rollback candidates rather than confirmed exploits. A full exploit would also require demonstrating that an attacker can control or compromise the provider dependency and that the downstream code path uses the vulnerable functionality. The results nevertheless show that the rollback model is not merely hypothetical: there are current install layouts in which a downstream package reaches a phantom through a transitive dependency, and the provider’s declared range can constrain the version available to downstream phantom imports.

4.2 The PyPI Ecosystem

The PyPI results are intentionally reported as static-analysis candidates. After filtering obvious artifacts, 593 of 801 successfully analyzed packages have at least one phantom candidate (Table 4). The most common candidate names include `typing-extensions`, `requests`, `numpy`, `packaging`, `ipython`, `jinja2`, `pyyaml`, and `pydantic-core`. Many of these are plausible runtime packages, but some can also appear through optional dependencies, compatibility code, or packaging-specific import conventions.

OSV advisory-history linkage finds 180 unique PyPI phantom candidates with at least one advisory, accounting for 1,072 static occurrences across 348 packages. We do not claim that these packages are currently vulnerable. The present PyPI collector does not resolve installed versions, evaluate markers and extras through `pip`’s resolver, or prove that a particular provider supplies the imported distribution. Consequently, the PyPI vulnerability numbers should be read as prioritization signals for future validation, not as vulnerability prevalence.

5 Discussion and Future Work

Our results characterize the scope of externally managed phantom dependencies in popular npm and PyPI packages and quantify several forms of downstream exposure. They also show why phantom dependency measurement has to be more precise than a raw manifest/static-analysis comparison. A package name that is reported as missing by a static tool may

be absent from the dependency closure, may be part of the same maintainer family, or may come from build-time and test-time code that is not relevant to runtime exposure. Treating these cases separately is essential for estimating prevalence and security impact.

The npm analysis is limited by `depcheck` coverage. About one third of package `depcheck` runs failed, and 415 seeds have at least one failed package in their closure. Our npm prevalence values are therefore lower bounds over the successfully analyzed portion of each closure.

The npm database is also name- and latest-version based. Resolver validation uses current npm installs for security-relevant seeds, which improves the rollback estimate, but the broad closure analysis does not reconstruct every historical package-lock tree. Monorepos and source repositories can also differ from published tarballs. A future pipeline should analyze exact npm tarballs or published `gitHead` commits and run static analysis in the package subdirectory specified by registry metadata.

For PyPI, the largest limitation is the absence of resolver-backed versions. Python import names do not always equal distribution names, optional dependencies and environment markers matter, and many packages use generated code or compatibility imports. A version-aware PyPI vulnerability analysis should use `pip install -dry-run -report`, map imports to installed distributions through metadata, and evaluate OSV ranges against resolved versions.

Several directions follow naturally. First, a fuller vulnerability assessment should distinguish build-time, test-time, optional, and runtime code paths and incorporate additional signals such as maintenance status and package abandonment. Second, the rollback attack model should be evaluated with end-to-end proof-of-concept cases that show how a transitive dependency can constrain the resolved version of a phantom package in practice. Third, the analysis should be scaled beyond the top 1,000 packages in each ecosystem to measure how phantom dependency behavior changes among less popular packages. Finally, the results raise an explanatory question: why do projects continue to use undeclared dependencies even though ecosystem-specific tools can often detect them? Understanding that maintenance behavior is necessary for turning measurement into effective developer guidance.

6 Conclusion

In this paper, we provide a large-scale analysis of phantom dependencies in npm and PyPI, two of the most widely used package registries for JavaScript and Python projects. We find that phantom dependencies are present among popular packages in both registries, but that their security interpretation depends strongly on resolver behavior and false-positive filtering. In npm, a strict closure- and family-filtered analysis of the top 1,000 packages finds 118 unique externally managed phantom packages and 567 seed-closure occurrences.

Resolver validation narrows the version-aware security conclusion further: 18 reachable rows across 5 seeds currently resolve to affected versions, and 15 strict rollback-candidate rows across 6 seeds are reachable in actual installs. In PyPI, filtered static analysis finds many candidates, but the security interpretation remains advisory-history only until resolver-backed version data is added. Overall, our results suggest that phantom dependencies are a practical supply-chain visibility problem, and that tooling integration can improve visibility, control, and remediation when the analysis is resolver-aware.

Availability

The accompanying repository contains the analysis code, generated CSV/JSON artifacts, and this LaTeX folder. The main reproducibility commands are documented in `README.md`. The generated artifacts used in this paper are copied into `paper_usenix/data/`. The PyPI claims use the `pypi-collected-*` artifacts.

References

- [1] Mahmoud Alfadel, Diego Elias Costa, and Emad Shihab. Empirical analysis of security vulnerabilities in python packages. *Empirical Software Engineering*, 28(3):59, 2023.
- [2] Peng Deng, Lei Zhang, Yuchuan Meng, Zhemin Yang, Yuan Zhang, and Min Yang. Chainfuzz: Exploiting upstream vulnerabilities in open-source supply chains. In *34th USENIX Security Symposium*, 2025.
- [3] depcheck contributors. depcheck. <https://github.com/depcheck/depcheck>, 2026. Used for JavaScript static dependency-use analysis.
- [4] Seth Michael Larson. PEP 770: Improving measurability of python packages with software bill-of-materials. Python Enhancement Proposals, 2025. Accessed 2026-05-24.
- [5] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *44th International Conference on Software Engineering*, pages 672–684, 2022.
- [6] Open Source Vulnerabilities. OSV: Open source vulnerability database. <https://osv.dev/>, 2026. Queried 2026-05-24.
- [7] Anand Sawant. Dependency resolution in python: Beware the phantom dependency. Endor Labs, 2023. Accessed 2026-05-24.
- [8] Xueqiang Wang, Yifan Zhang, XiaoFeng Wang, Yan Jia, and Luyi Xing. Union under duress: Understanding hazards of duplicate resource mismediation in android software supply chain. In *32nd USENIX Security Symposium*, 2023.
- [9] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. On the impact of security vulnerabilities in the npm and rubygems dependency networks. *Empirical Software Engineering*, 27(5):107, 2022.
- [10] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. Small world with high risks: A study of security threats in the npm ecosystem. In *28th USENIX Security Symposium*, 2019.